Contract Lecturer Monthly Claims System

ST10445832

Alex Pluddemann

PROG6212

CMCS Purpose



To allow contract lecturers to easily make claims to the institution for work carried out.



To facilitate transparency in the claims process.



To increase claim processing efficiency.



To provide actionable insights to HR.

CMCS Users

HR

- Process approved claims & generate invoices
- Generate system usage reports
- Onboard users and update existing user information

Academic Manager

- Review and approve verified claims

Programme Coordinator

- Review and verify lodged claims

Lecturer

- Submit claims

Problem



Complex claims process.



Inefficient approval process.



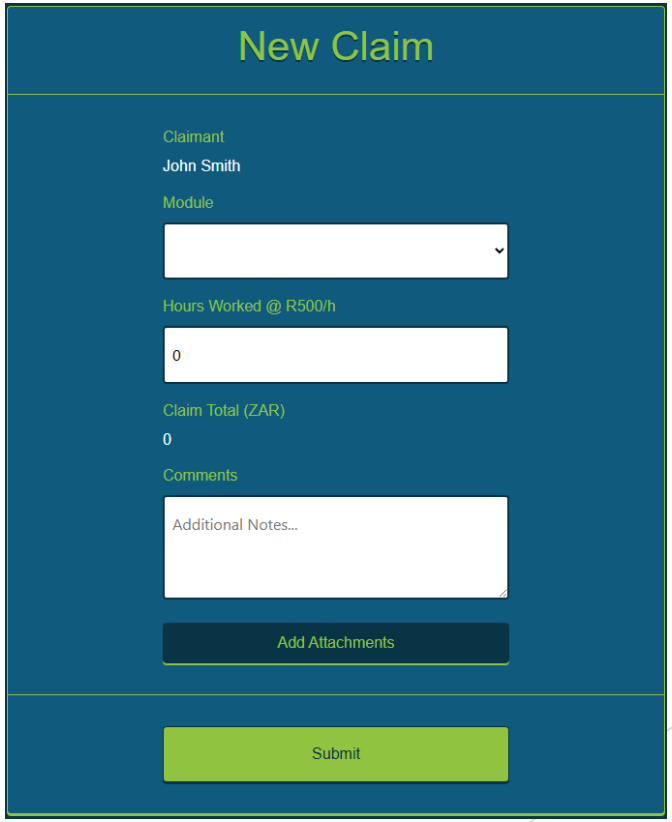
Repetitive data entry.



Manual invoice creation.

Solution: Simplified Claim Submission Limited Module Options

- Minimum 2 inputs, Maximum 4 inputs
- Lecturer need only provide
  - The module relevant to their claim
  - Hours worked
  - (optional) Comments and supporting documents which might help in verifying the validity of the claim



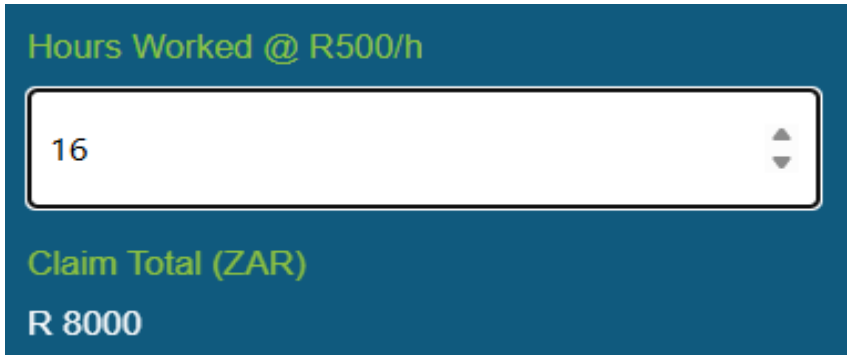
Solution: Pre-validated module selection

- Simple
  - Lecturers only see modules they teach, and not all modules offered by the institution
- Safe
  - Lecturers cannot claim against modules which they don’t teach

```
public async Task<List<Module>> GetLecturerModulesAsync(string lecturerID)
{
    return await _dbContext.Modules
        .Join(_dbContext.CourseModules,
            m => m.ModuleID,
            cm => cm.ModuleID,
            (m, cm) => new { module = m, cm.CourseModuleID })
        .Join(_dbContext.LecturerModules.Where(lm => lm.LecturerID.Equals(lecturerID)),
            mcm => mcm.CourseModuleID,
            lm => lm.CourseModuleID,
            (mcm, lm) => new Module
            {
                ModuleID = mcm.module.ModuleID,
                ModuleCode = mcm.module.ModuleCode,
                ModuleName = mcm.module.ModuleName,
            })
        .ToListAsync();
}
```

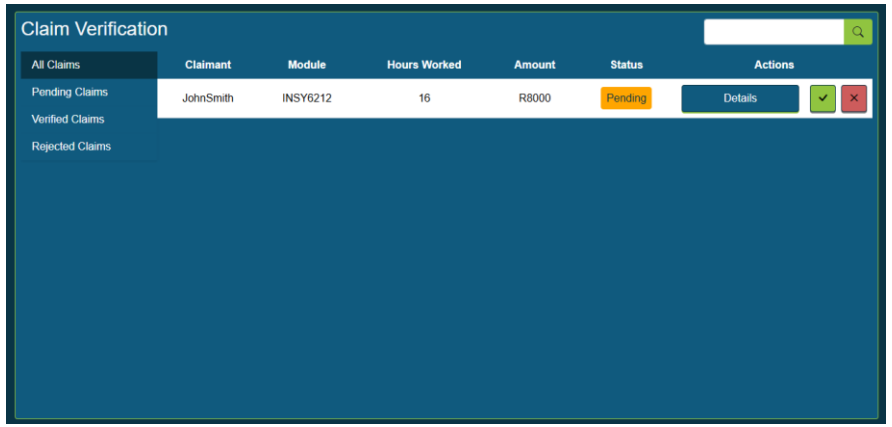
Solution: Simplified claim valuation

- Lecturers' hourly rate
  - Set by HR during onboarding
  - Pulled from database during claim creation
- Claim value auto-calculates using stored hourly rate and entered hours worked
- Saves time for both claim and verification
- Reduces manual verification tasks

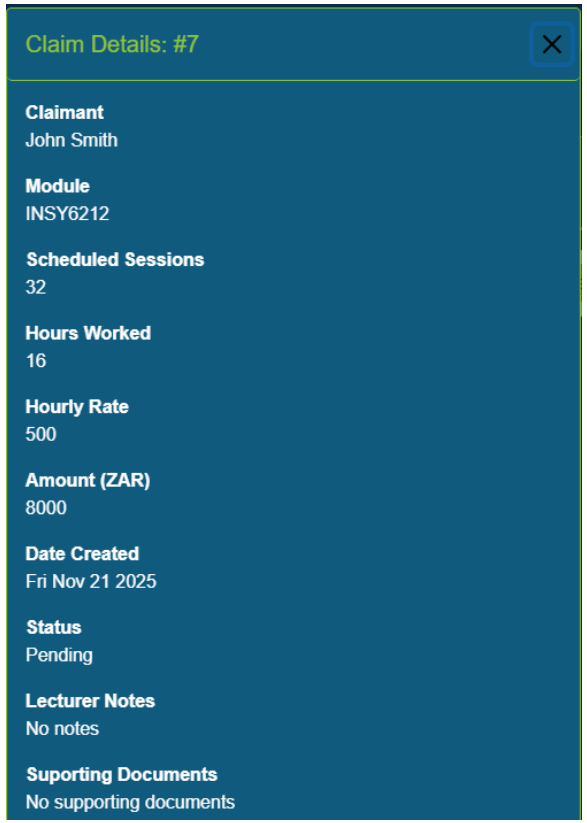



```
// auto calculate claim amount using saved hourly rate
const hoursWorkedInput = document.getElementById('hours-worked-input');
hoursWorkedInput.addEventListener('input', () => {
    var hoursWorked = parseFloat(hoursWorkedInput.value) || 0; // zero if invalid
    var hourlyRate = @Model.HourlyRate;
    var claimTotal = hoursWorked * hourlyRate;
    // display value
    document.getElementById('claim-total-display').innerText = 'R ' + claimTotal;
});
```

Solution: simplified approval



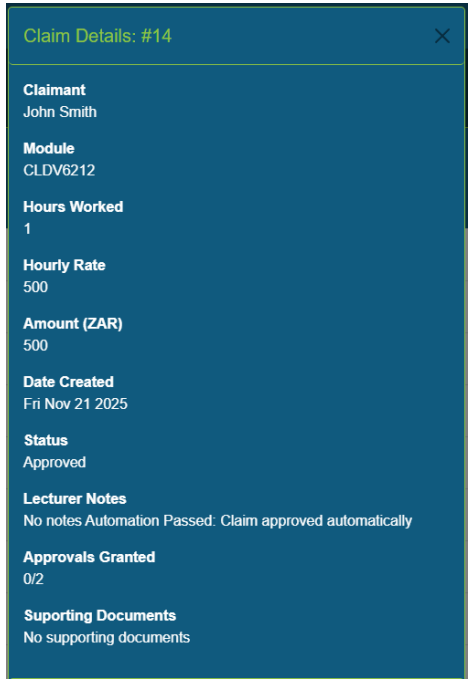
- Approval can be granted based on data at a glance
- Claim detail can be viewed, including attached documents, if further scrutiny is required.

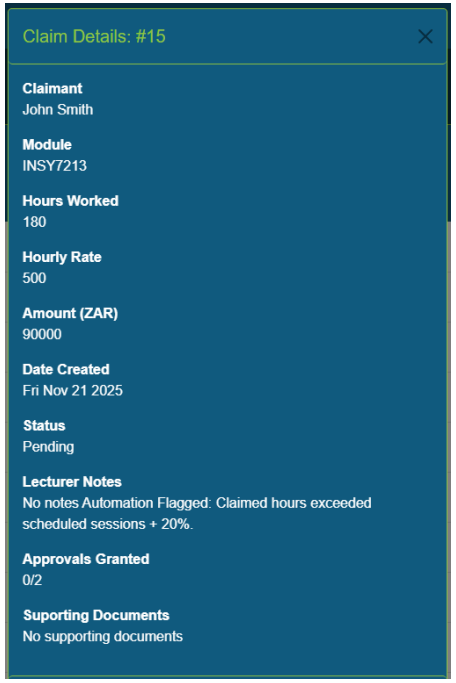


```
// hourly rate must match lecturer rate
if (!claim.HourlyRate.Equals((await _repository.GetUserByIdAsync(claim.LecturerID)).HourlyRate))
{
    return (false, "Automation Flagged: Hourly rate mismatch.");
}

// claimed hours (month total) must not exceed scheduled sessions + 20%
var date = DateOnly.FromDateTime(DateTime.Now);
(double, int) hoursClaimedAndSessions = await _repository.GetMonthHoursClaimedAndSessions(claim.CourseModuleID, date);
if ((hoursClaimedAndSessions.Item1 + claim.HoursWorked) >
    ((double)hoursClaimedAndSessions.Item2 + ((double)hoursClaimedAndSessions.Item2 * 0.2)))
{
    return (false, "Automation Flagged: Claimed hours exceeded scheduled sessions for this month + 20%.");
}
// total claimed hours for the month must not exceed 180 hours
if (hoursClaimedAndSessions.Item1 + claim.HoursWorked > 180)
{
    return (false, "Automation Flagged: Monthly claimd hours exceeded limit of 180 hours.");
}

return (true, "Automation Passed: Claim approved automatically");
```



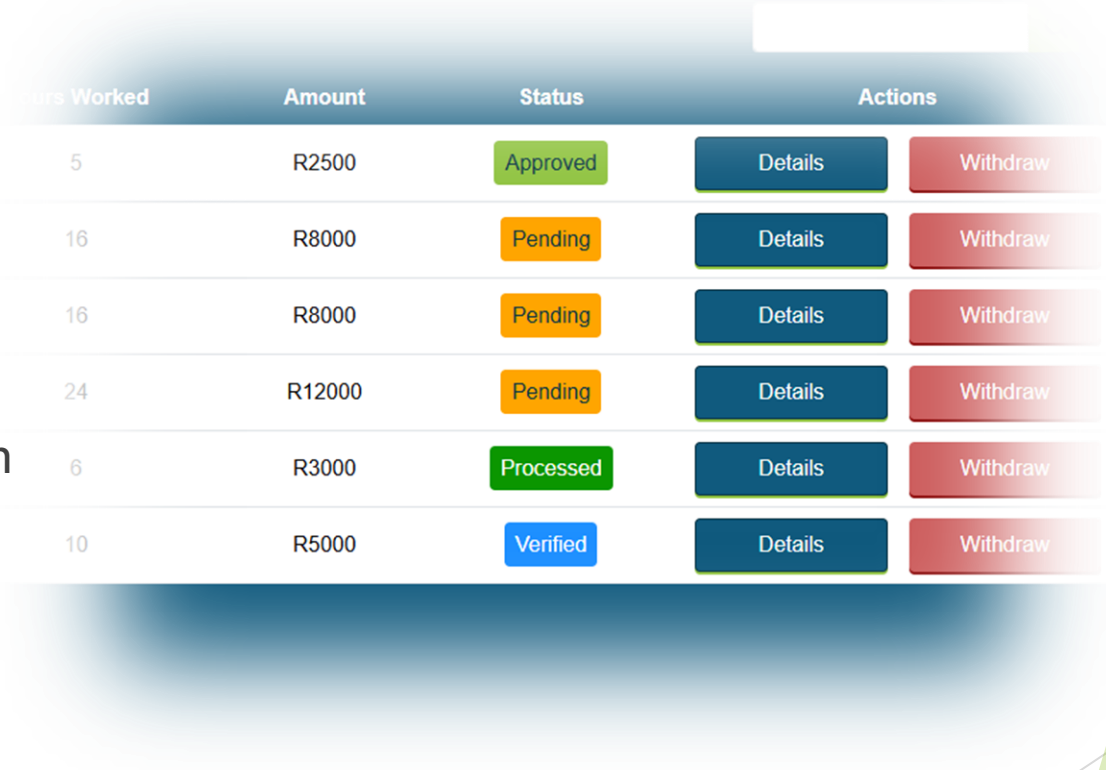


Solution: automated approval

- The system can auto-approve claims that fall within certain criteria.

Solution: transparent approval process

- Claim status is updated at each step of the process
- Status is updated in real-time, allowing the lecturer to see updates as they happen
- Reduces queries regarding claim approvals



urs Worked	Amount	Status	Actions	
5	R2500	Approved	Details	Withdraw
16	R8000	Pending	Details	Withdraw
16	R8000	Pending	Details	Withdraw
24	R12000	Pending	Details	Withdraw
6	R3000	Processed	Details	Withdraw
10	R5000	Verified	Details	Withdraw

Real-time status updates with SignalR

Lecturer View

1. Client connects to SignalR Hub

2. Define callback function for when ‘ClaimStatusUpdated’ triggers

3. When callback function runs, update the UI for the relevant claim

```
const conn = new signalR.HubConnectionBuilder()
    .withUrl('/StatusHub')
    .withAutomaticReconnect()
    .build();

conn.on('ClaimStatusUpdated', (claimID, statusTitle) => {
    const claimRow = document.querySelector(`[data-claim-id="${claimID}"]`);



statusSpan.innerText = statusTitle;

else if (statusTitle === "Verified")
{
    statusSpan.classList.remove('pending-status');
    statusSpan.classList.remove('approved-status');
    statusSpan.classList.remove('rejected-status');
    statusSpan.classList.add('verified-status');
}
```

Approval Controller

1. Send status to update notice method when claim is verified

2. SignalR hub notifies clients of the event

```
// send update notification for real-time update
await _listener.NotifyOnStatusUpdate(claim.ClaimID, "Verified");

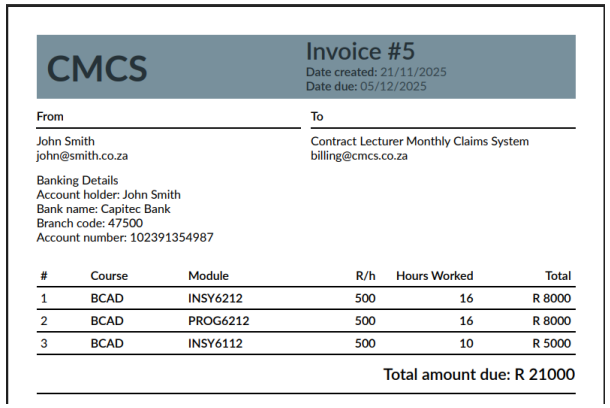

4 references
public async Task NotifyOnStatusUpdate(int claimID, string statusTitle)
{
    await _hubContext.Clients.All.SendAsync("ClaimStatusUpdated", claimID, statusTitle);
}
```

Solution: automated invoice generation

Process Claims

Claimant	Module Code	Hours Worked	Claim Value	Status	Date Created	Actions
John Smith	INSY6212	16	8000	Approved	2025/11/21	Deselect Process Reject
John Smith	PROG6212	16	8000	Approved	2025/11/21	Deselect Process Reject
John Smith	PROG6211	5	2500	Approved	2025/11/21	Select Process Reject
John Smith	INSY6112	10	5000	Approved	2025/11/21	Deselect Process Reject
John Smith	CLDV6212	24	12000	Approved	2025/11/21	Select Process Reject

HR can select one or more claims to process. This completes the claim process and automatically generates an invoice, stored in the system database.



CMCS

Invoice #5
Date created: 21/11/2025
Date due: 05/12/2025

From
John Smith
john@smith.co.za

To
Contract Lecturer Monthly Claims System
billing@cmcs.co.za

Banking Details
Account holder: John Smith
Bank name: Capitec Bank
Branch code: 47500
Account number: 102391354987

#	Course	Module	R/h	Hours Worked	Total
1	BCAD	INSY6212	500	16	R 8000
2	BCAD	PROG6212	500	16	R 8000
3	BCAD	INSY6112	500	10	R 5000

Total amount due: R 21000

Invoice Generation

- Claim and relevant lecturer data is read from the database
- The data fed to a QuestPDF Document class which defines the invoice template
- The document content is saved to the database with information like invoice number, date created, and due date.

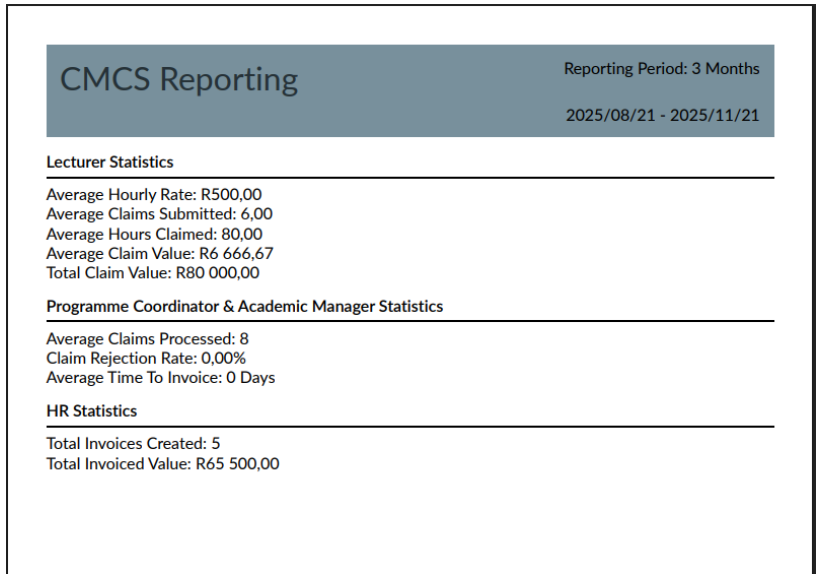
```
// relevant claim information
var claims = await _dbContext.Claims.Where(claim => ClaimIDs.Contains(claim.ClaimID))
    .Include(claim => claim.Lecturer)
        .ThenInclude(lecturer => lecturer.BankAccount)
    .Include(claim => claim.LecturerModule.CourseModule.Course)
    .Include(claim => claim.LecturerModule.CourseModule.Module)
    .ToListAsync();
//var claimant = await _userManager.FindByIdAsync(claims.First().LecturerID);
var claimant = claims.Select(claim => claim.Lecturer).First();

invoiceDetail.InvoiceeName = $"{claimant.FirstName} {claimant.LastName}";
invoiceDetail.InvoiceeEmail = claimant.Email;
invoiceDetail.BankAccount = claimant.BankAccount;
invoiceDetail.TotalAmount = claims.Select(claim => claim.HourlyRate * claim.HoursWorked).Sum();

var invoiceItems = new List<InvoiceItem>();

foreach (var claim in claims)
{
    invoiceItems.Add(new InvoiceItem
    {
        CourseCode = claim.LecturerModule.CourseModule.Course.CourseCode,
        ModuleCode = claim.LecturerModule.CourseModule.Module.ModuleCode,
        HourlyRate = claim.HourlyRate,
        HoursWorked = claim.HoursWorked,
        Total = claim.HourlyRate * claim.HoursWorked
    });
}

// generate invoice as pdf
var invoice = new InvoiceDocument(invoiceDetail);
// save invoice to DB
byte[] invoiceBytes = invoice.GeneratePdf();
Invoice newInvoice = new Invoice
{
    InvoiceID = invoice.InvoiceDetail.InvoiceNumber,
    Content = invoiceBytes,
    PaymentDue = invoice.InvoiceDetail.PaymentDue,
    DateCreated = invoice.InvoiceDetail.DateCreated
};
```



CMCS Reporting

Reporting Period: 3 Months

2025/08/21 - 2025/11/21

Lecturer Statistics

Average Hourly Rate: R500,00
Average Claims Submitted: 6,00
Average Hours Claimed: 80,00
Average Claim Value: R6 666,67
Total Claim Value: R80 000,00

Programme Coordinator & Academic Manager Statistics

Average Claims Processed: 8
Claim Rejection Rate: 0,00%
Average Time To Invoice: 0 Days

HR Statistics

Total Invoices Created: 5
Total Invoiced Value: R65 500,00

Solution: User and business insights

- HR can generate statistics report documents for
  - All time
  - 3 Months
  - 6 Months
  - 12 Months

Solution: User and business insights

- Lecturer Statistics
  - Average hourly rate
  - Average hours claimed
  - Average number of claims per lecturer
  - Average claim value

```
// Average hourly rate of all lecturers
var averageHourlyRate = lecturers.Count == 0 ? 0.0 : await _dbContext.Users
    .Where(user => lecturers.Contains(user))
    .AverageAsync(user => user.HourlyRate);

// average number of claims submitted per lecturer
var averageClaims = await claimsQueryable.CountAsync() == 0 ? 0.0 : await claimsQueryable
    .GroupBy(claim => claim.LecturerID)
    .Select(group => group.Count())
    .AverageAsync();

// average value per claim
var averageClaimValue = await claimsQueryable.CountAsync() == 0 ? 0.0 : await claimsQueryable
    .AverageAsync(claim => claim.HourlyRate * claim.HoursWorked);
```

Statistics gathering example

- Claim Statistics
  - Rejection rate
  - Average time from claim submission to processing completion
  - Total claims processed
  - Total number of invoices
  - Total amount invoiced

Solution: User and business insights

- Further insights possible
  - Per approver productivity metrics
  - Lecturer rates per module/course
  - Cost efficiency suggestions
  - Module pricing guidance based on lecturer cost

Conclusion

- The CMCS
  - Empowers lecturers
  - Enhances institution productivity and operational efficiency
  - Delivers actionable business insights